

EXP : 4

```
import numpy as np

# Sigmoid Activation Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Input dataset (XOR problem)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Output dataset
y = np.array([[0], [1], [1], [0]])

# Seed for reproducibility
np.random.seed(1)

# Initialize weights randomly
input_neurons = 2
hidden_neurons = 4
output_neurons = 1
```

```

# Weights
weights_input_hidden = np.random.uniform(size=(input_neurons, hidden_neurons))
weights_hidden_output = np.random.uniform(size=(hidden_neurons, output_neurons))

# Bias
bias_hidden = np.random.uniform(size=(1, hidden_neurons))
bias_output = np.random.uniform(size=(1, output_neurons))

# Learning rate
lr = 0.5

# Training the network
epochs = 10000

for epoch in range(epochs):

    # ----- Forward Propagation -----
    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, weights_hidden_output) + bias_output
    predicted_output = sigmoid(final_input)

    # ----- Backpropagation -----
    error = y - predicted_output

    d_predicted_output = error * sigmoid_derivative(predicted_output)

    error_hidden_layer = d_predicted_output.dot(weights_hidden_output.T)
    d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_output)

```

```
# ----- Update Weights -----
```

```
weights_hidden_output += hidden_output.T.dot(d_predicted_output) * lr
```

```
weights_input_hidden += X.T.dot(d_hidden_layer) * lr
```

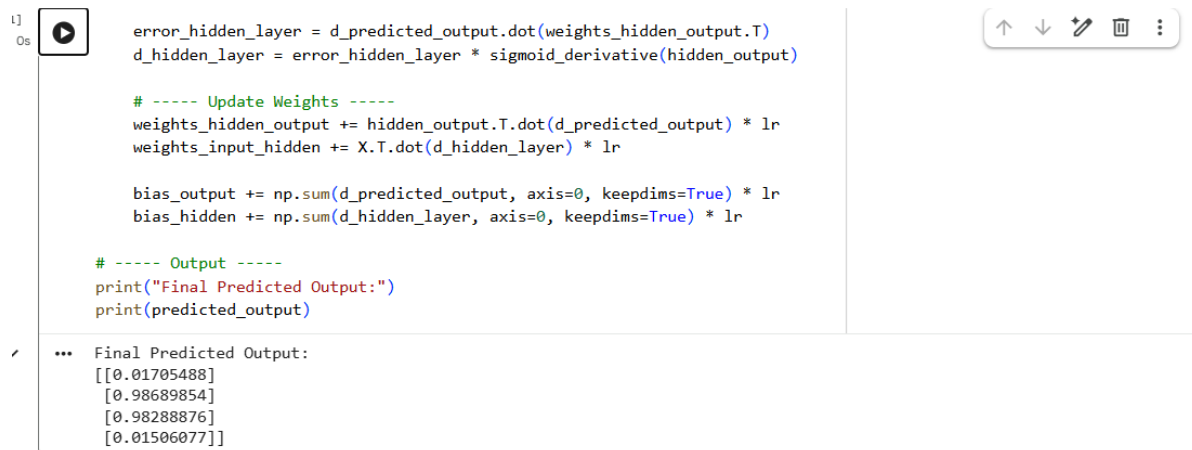
```
bias_output += np.sum(d_predicted_output, axis=0, keepdims=True) * lr
```

```
bias_hidden += np.sum(d_hidden_layer, axis=0, keepdims=True) * lr
```

```
# ----- Output -----
```

```
print("Final Predicted Output:")
```

```
print(predicted_output)
```



The image shows a Jupyter Notebook interface. On the left, there is a vertical toolbar with a play button icon. The main area contains Python code for a neural network's backpropagation and output calculation. The code includes comments for 'Update Weights' and 'Output'. The output of the final predicted output is displayed as a list of four arrays.

```
error_hidden_layer = d_predicted_output.dot(weights_hidden_output.T)
d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_output)

# ----- Update Weights -----
weights_hidden_output += hidden_output.T.dot(d_predicted_output) * lr
weights_input_hidden += X.T.dot(d_hidden_layer) * lr

bias_output += np.sum(d_predicted_output, axis=0, keepdims=True) * lr
bias_hidden += np.sum(d_hidden_layer, axis=0, keepdims=True) * lr

# ----- Output -----
print("Final Predicted Output:")
print(predicted_output)
```

Final Predicted Output:

```
[[0.01705488]
 [0.98689854]
 [0.98288876]
 [0.01506077]]
```